

bok25

**基
础**

班、计算机组成原理

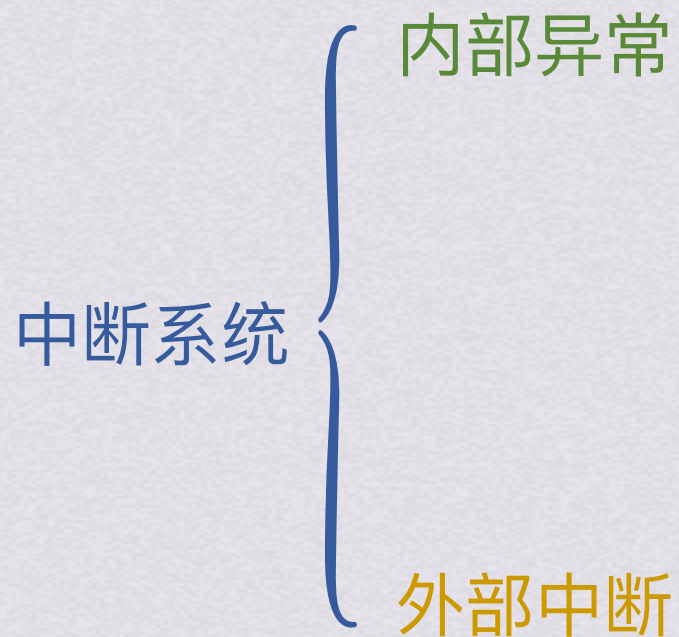


异常和中断



异常和中断处理

程序正常执行过程中，CPU会遇到一些特殊情况而无法继续执行当前程序，可分为两种情况





异常和中断处理

系统

内部异常

指由处理器内部异常引起的意外事件

终止

由硬连线路出现异常引起的
如电源掉电，存储器线路错等

故障

自陷

硬故障中断



外部中断

异常和中断处理

统

内部异常

指由处理器内部异常引起的意外事件

终止

由硬连线路出现异常引起的
如电源掉电，存储器线路错等

故障

也称失效，是在引起故障的指令
启动后、执行结束前被检测到的一
类异常事件
如保护错、整除0、非法操作
码、缺页、段不存在

自陷

外部中断

$$\frac{x}{0}$$

int a = 2,147,483,647 * 2,147,483,647



异常和中断处理

统

内部异常

指由处理器内部异常引起的意外事件

终止

由硬连线路出现异常引起的
如电源掉电，存储器线路错等

故障

也称失效，是在引起故障的指令
启动后、执行结束前被检测到的一
类异常事件

如保护错、整除0、非法操作
码、

自陷

缺页、段不存在

这类异常处理后，已将需要的段或页面从磁盘调入主存，所以可
以继续回到发生故障的指令重新执行
断点位当前发生故障的指令

外部中断



异常和中断处理

统

内部异常

指由处理器内部异常引起的意外事件

终止

由硬连线路出现异常引起的
如电源掉电，存储器线路错等

故障

也称失效，是在引起故障的指令
启动后、执行结束前被检测到的一
类异常事件

自陷

如**保护错**、**整除0**、
非法操作码、缺页、
段不存在

外部中断

这类异常处理后，已将需要的段或
页面从磁盘调入主存，所以可以继续
回到发生故障的指令重新执行
断点位当前发生故障的指令

对于这些故障，因为无法通过异常
处理程序恢复故障，因此不能回到
原断点继续执行，必须终止进程的
执行；

对于浮点除法“除数位0”，有不同
处理方式，可将指令执行结果用特
殊值 (∞ 或NaN) 来表示后继续回
原断点执行，也可以终止进程的执
行
(NaN = Not a Number)



异常和中断处理

统 {

内部异常

指由处理器内部异常引起的意外事件

外部中断

终止

由硬连线路出现异常引起的
如电源掉电，存储器线路错等

故障

也称失效，是在引起故障的指令启动后、执行结束前被检测到的一类异常事件
如保护错、整除0、非法操作码、缺页、段不存在

自陷

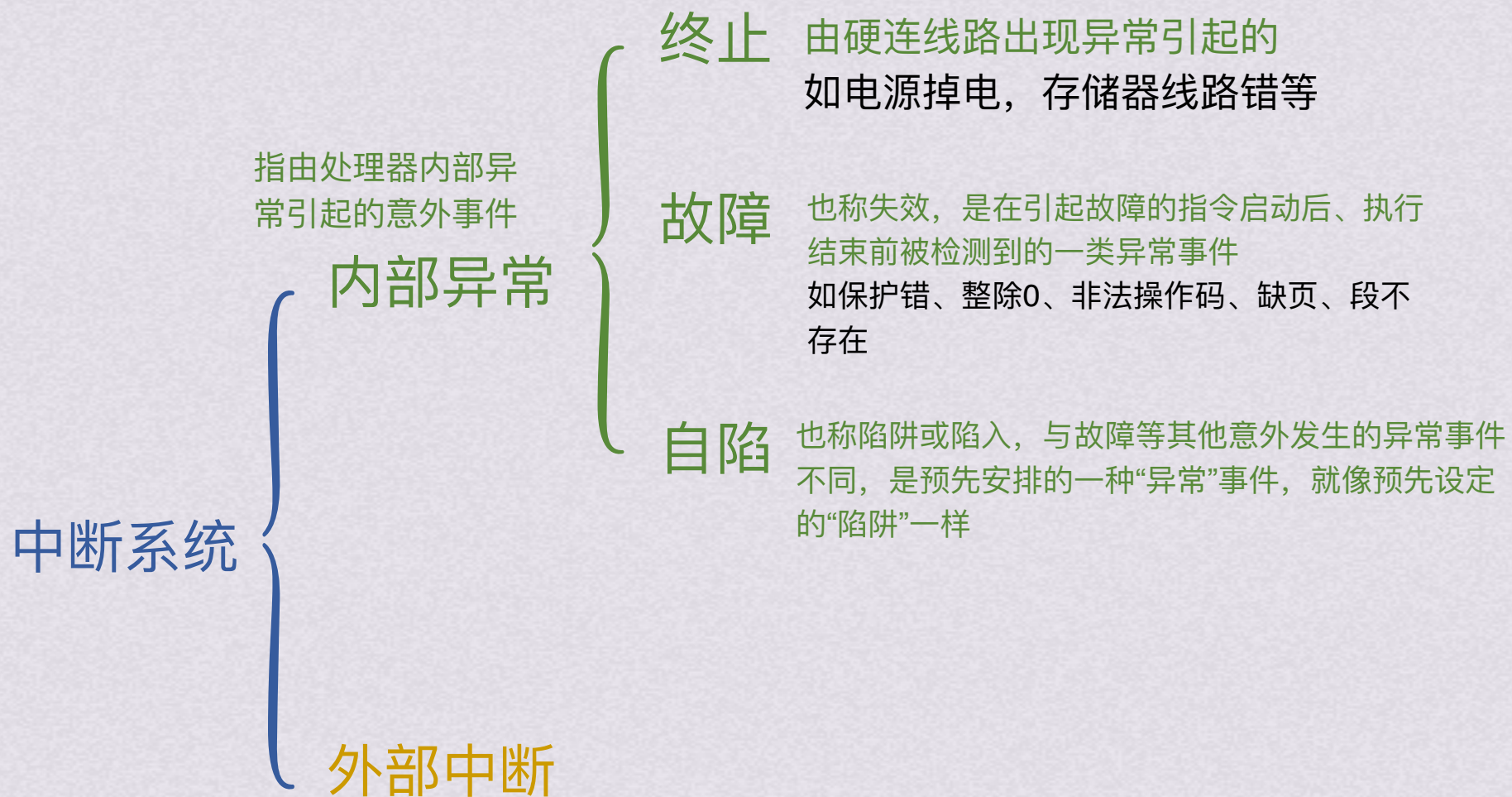
也称陷阱或陷入，与故障等其他意外发生的异常事件不同，是预先安排的一种“异常”事件，就像预先设定的“陷阱”一样，首先通过某种方式将CPU设定为处于某个特定状态，程序执行过程中，一旦某条指令的执行发生了相应状态所满足的条件，则CPU调出特定的程序进行相应处理

还有系统调用指令、条件自陷指令等都属于陷阱指令，执行到这些指令时，无条件或有条件地自动调出操作系统内核程序进行执行

*注：自陷在《操作系统》这门课程中还会详细解释



异常和中断处理





异常和中断处理

中断系统

内部异常

结束前被检测到的一类异常事件

如保护错、整除0、非法操作码、缺页、段不存在

自陷

也称陷阱或陷入，与故障等其他意外发生的异常事件不同，是预先安排的一种“异常”事件，就像预先设定的“陷阱”一样

外部中断

因为这些事件和执行的指令无关，由CPU外部的IO发出，所以，称为IO中断或外部中断，需要通过外部中断请求线向CPU请求

程序执行过程中，若外设完成任务或发生某些特殊事件（打印机缺纸、键盘缓冲满等），会向CPU发中断请求，要求CPU对这些情况进行处理。

通常每条指令执行完后，CPU都会主动去查询有没有中断请求，有的话，将**下一条指令地址**作为断点保存，然后转到相应的中断服务程序执行，结束后回到断点继续执行

CPU



指令1

指令2

指令3

指令4



异常和中断处理

中断系统

内部异常

结束前被检测到的一类异常事件

如保护错、整除0、非法操作码、缺页、段不存在

自陷

也称陷阱或陷入，与故障等其他意外发生的异常事件不同，是预先安排的一种“异常”事件，就像预先设定的“陷阱”一样

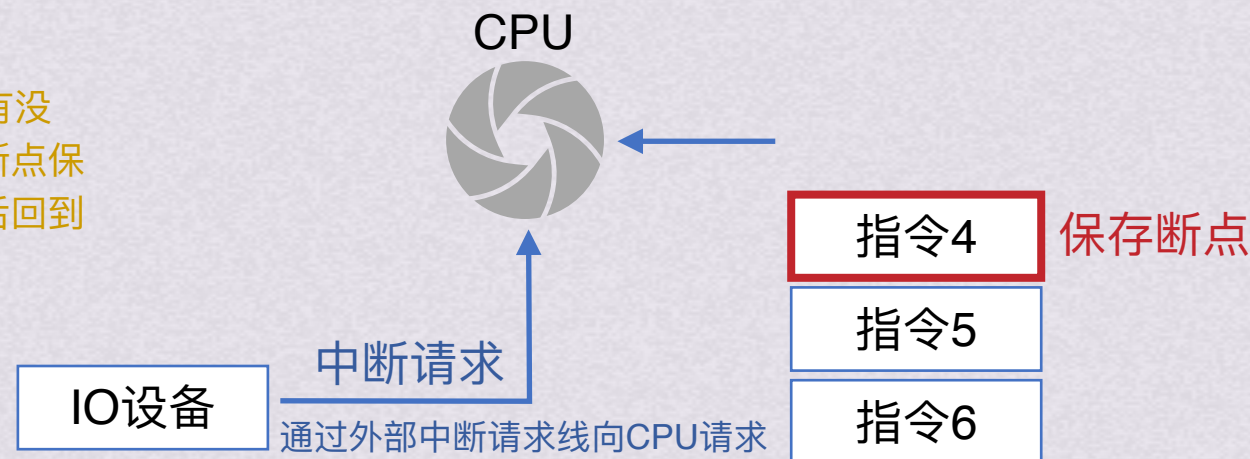
外部中断

因为这些事件和执行的指令无关，由CPU外部的IO发出，所以，称为IO中断或外部中断，需要通过外部中断请求线向CPU请求

程序执行过程中，若外设完成任务或发生某些特殊事件（打印机缺纸、键盘缓冲满等），会向CPU发中断请求，要求CPU对这些情况进行处理。

通常每条指令执行完后，CPU都会主动去查询有没有中断请求，有的话，将**下一条指令地址**作为断点保存，然后转到相应的中断服务程序执行，结束后回到断点继续执行

通过中断服务程序解决中断请求后回到断点继续执行





异常和中断处理

中断系统

内部异常

结束前被检测到的一类异常事件

如保护错、整除0、非法操作码、缺页、段不存在

自陷

也称陷阱或陷入，与故障等其他意外发生的异常事件不同，是预先安排的一种“异常”事件，就像预先设定的“陷阱”一样

外部中断

因为这些事件和执行的指令无关，由CPU外部的IO发出，所以，称为IO中断或外部中断，需要通过外部中断请求线向CPU请求

程序执行过程中，若外设完成任务或发生某些特殊事件（打印机缺纸、键盘缓冲满等），会向CPU发中断请求，要求CPU对这些情况进行处理。

通常每条指令执行完后，CPU都会主动去查询有没有中断请求，有的话，将**下一条指令地址**作为断点保存，然后转到相应的中断服务程序执行，结束后回到断点继续执行

通过中断服务程序解决中断请求后回到断点继续执行

IO设备的中断被解决后也继续工作

IO设备

CPU



指令4

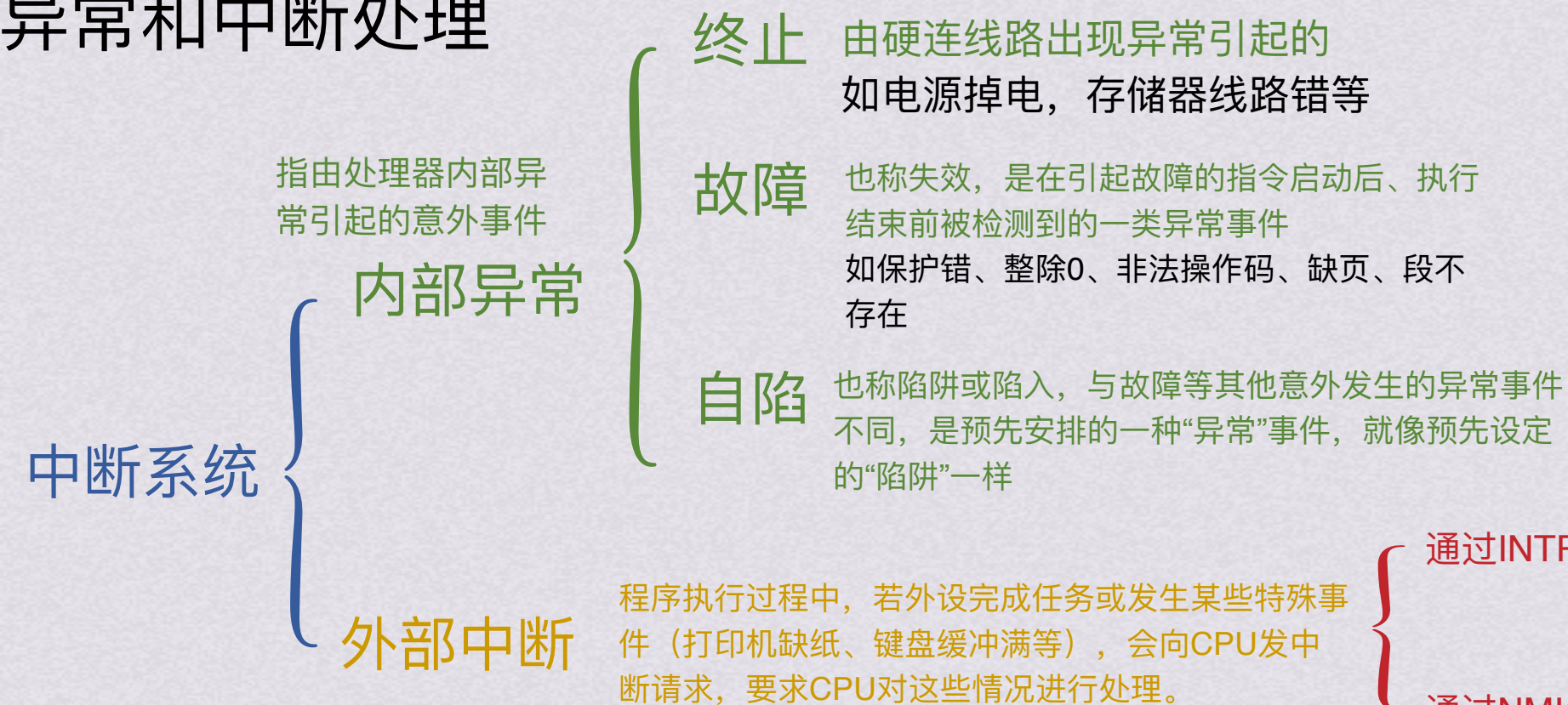
指令5

指令6

指令7



异常和中断处理



通过INTR信号线发出的外中断是可屏蔽中断

通过NMI信号发出的是不可屏蔽中断

INTR(INTeRrupt,可屏蔽中断)

NMI(NonMaskableInterrupt,不可屏蔽中断)

不可屏蔽中断的处理优先级最高，任何时候发生不可屏蔽中断，都要中止现行程序的执行，转到不可屏蔽中断处理程序执行

每个中断都有一个类型号，每个中断类型号都对应一个中断服务程序，其入口地址放在一个专门的中断向量表中



异常和中断处理

异常的处理过程

1.保护断点和程序状态

2.识别异常事件并转异常处理



异常和中断处理

异常的处理过程

1.保护断点和程序状态

对不同的异常事件，其返回地址（断点）不同

指由处理器内部异常引起的意外事件

内部异常

终止

由硬连线路出现异常引起的
如电源掉电，存储器线路错等

故障

也称失效，是在引起故障的指令启动后、执行
结束前被检测到的一类异常事件
如保护错、整除0、非法操作码、缺页、段不
存在

故障的断点是发生故障的当前指令的地址

自陷

也称陷阱或陷入，与故障等其他意外发生的异常事件
不同，是预先安排的一种“异常”事件，就像预先设定
的“陷阱”一样

自陷的断点是自陷指令后面一条指令的地址

断点的值由异常类型和发生异常时PC的值决定

为了能在异常处理后正确返回到原被中断程序继续执行，数据通路必须能正确计算断点值

假定计算出的断点值存放在PC中，则保护断点时，只要将PC送到栈或特定的寄存器中即可
(大多处理器将断点保存在栈中)



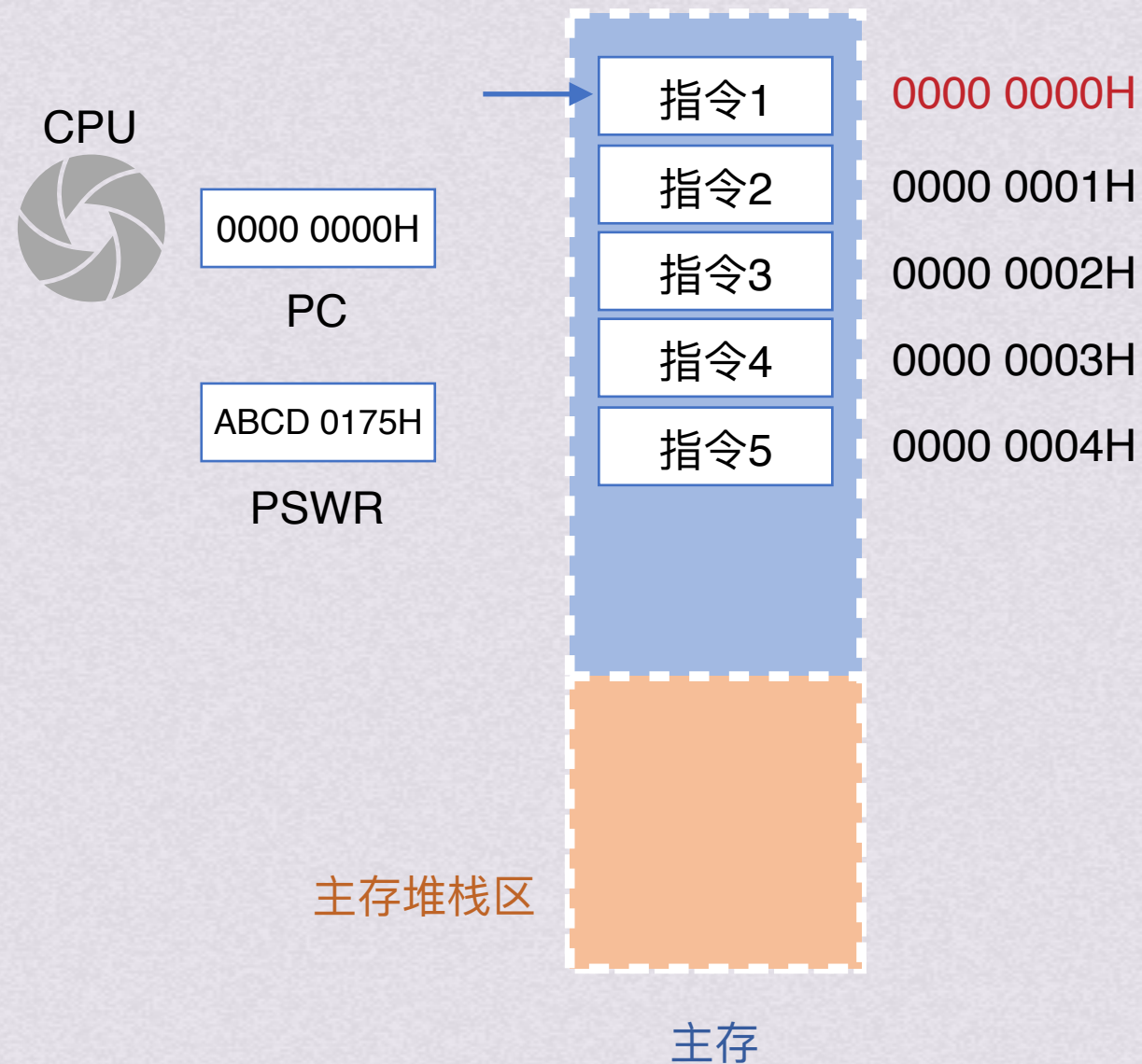
异常和中断处理

异常的处理过程

因为异常处理后可能还要回到原本被中断的程序继续执行，被中断时原程序的状态（如产生的各种标志信息、允许自陷标志等）都必须保存起来。

通常每个正在运行程序的状态信息被放在程序状态字寄存器（PSWR）中，存放在PSWR中的信息称为程序状态字PSW

与返回地址（断点）一样，PSW也要被保存到栈或特定寄存器中，在异常返回时，将保存的PSW恢复到PSWR中





异常和中断处理

异常的处理过程

因为异常处理后可能还要回到原本被中断的程序继续执行，被中断时原程序的状态（如产生的各种标志信息、允许自陷标志等）都必须保存起来。

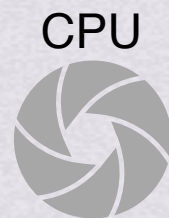
通常每个正在运行程序的状态信息被放在程序状态字寄存器（PSWR）中，存放在PSWR中的信息称为程序状态字PSW

与返回地址（断点）一样，PSW也要被保存到栈或特定寄存器中，在异常返回时，将保存的PSW恢复到PSWR中

缺页异常

转去处理缺页异常程序

保存断点+程序状态



0000 0001H

PC

ACCA 0106H

PSWR



主存堆栈区

主存

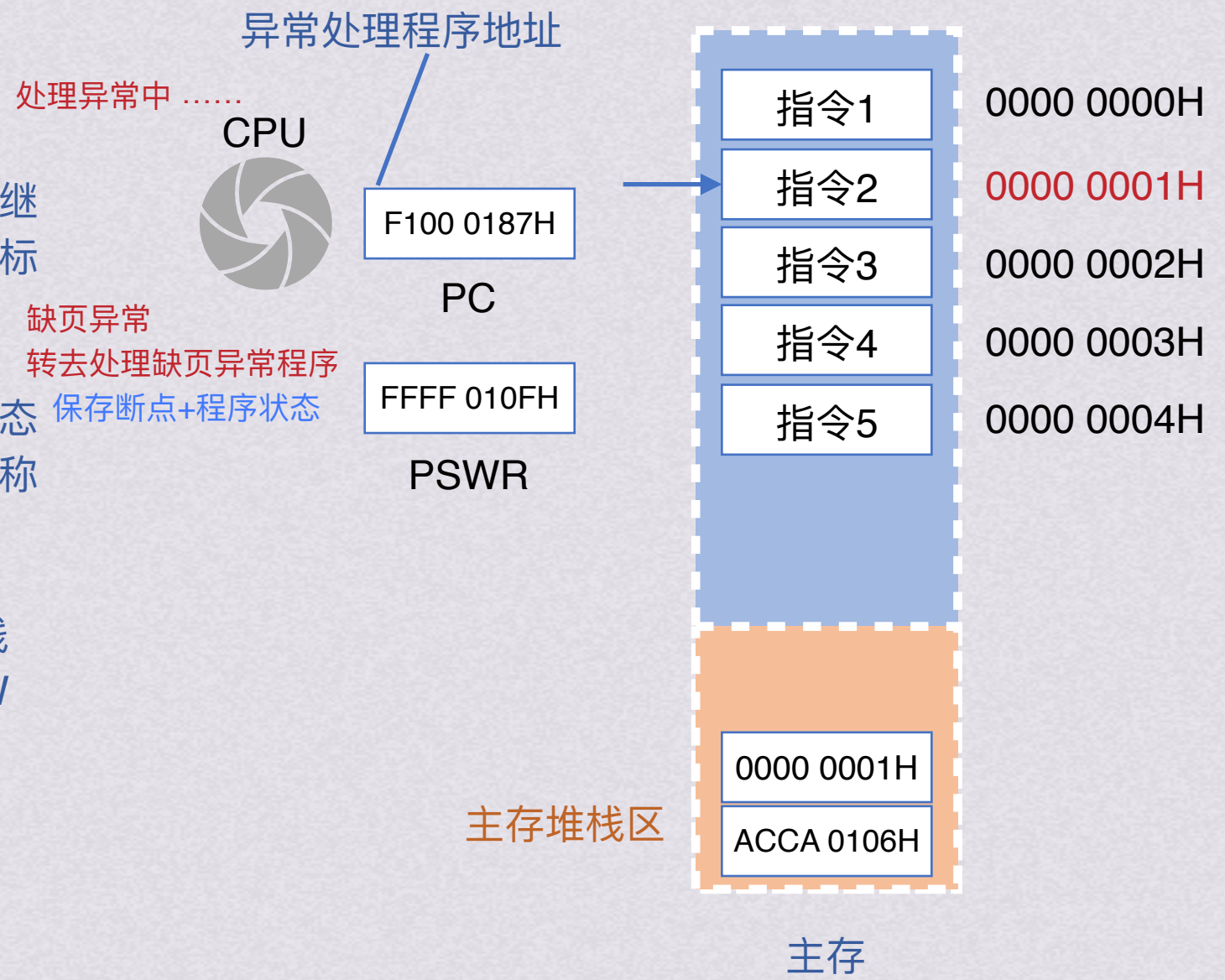
异常和中断处理

异常的处理过程

因为异常处理后可能还要回到原本被中断的程序继续执行，被中断时原程序的状态（如产生的各种标志信息、允许自陷标志等）都必须保存起来。

通常每个正在运行程序的状态信息被放在程序状态字寄存器（PSWR）中，存放在PSWR中的信息称为程序状态字PSW

与返回地址（断点）一样，PSW也要被保存到栈或特定寄存器中，在异常返回时，将保存的PSW恢复到PSWR中





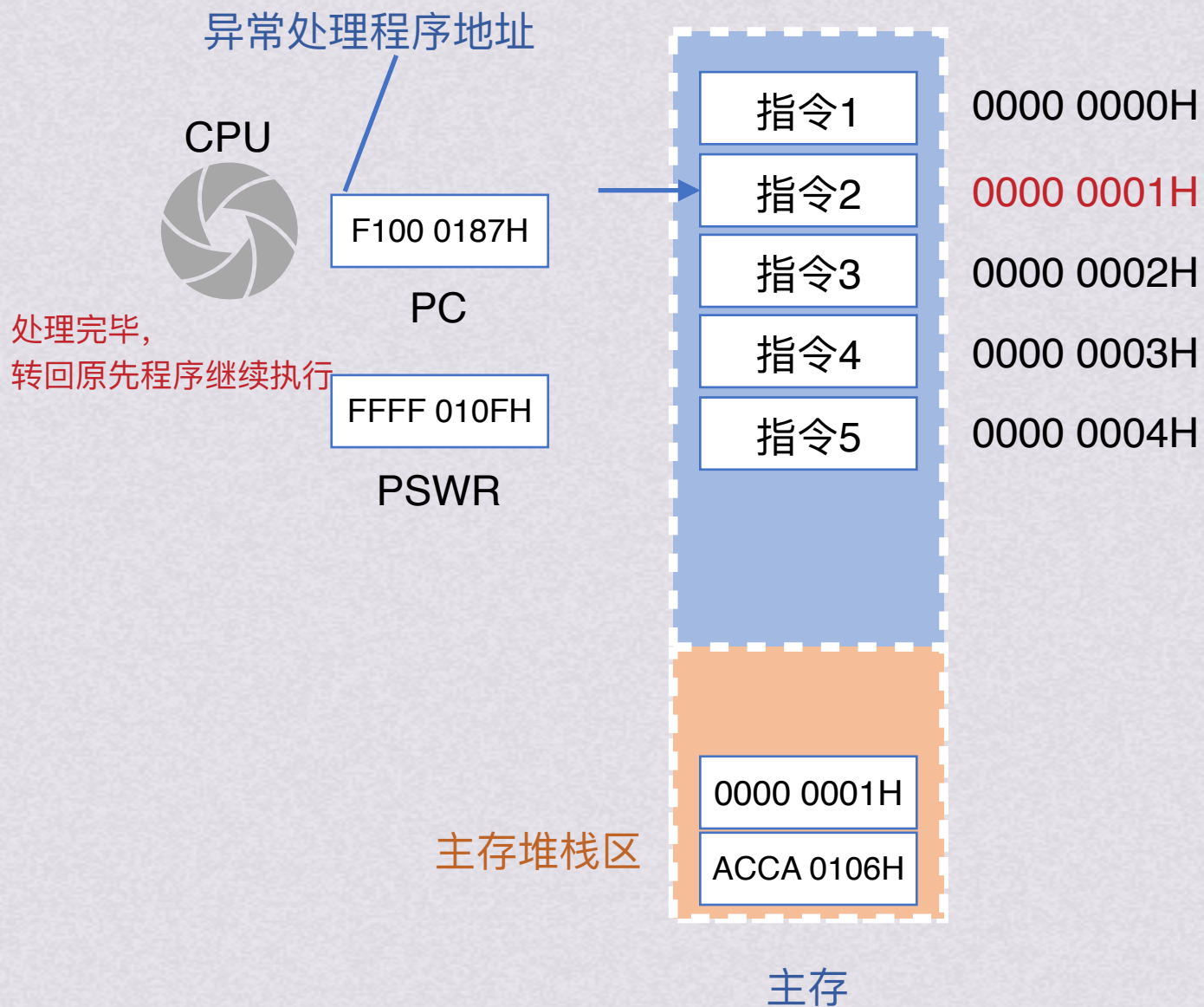
异常和中断处理

异常的处理过程

因为异常处理后可能还要回到原本被中断的程序继续执行，被中断时原程序的状态（如产生的各种标志信息、允许自陷标志等）都必须保存起来。

通常每个正在运行程序的状态信息被放在程序状态字寄存器（PSWR）中，存放在PSWR中的信息称为程序状态字PSW

与返回地址（断点）一样，PSW也要被保存到栈或特定寄存器中，在异常返回时，将保存的PSW恢复到PSWR中





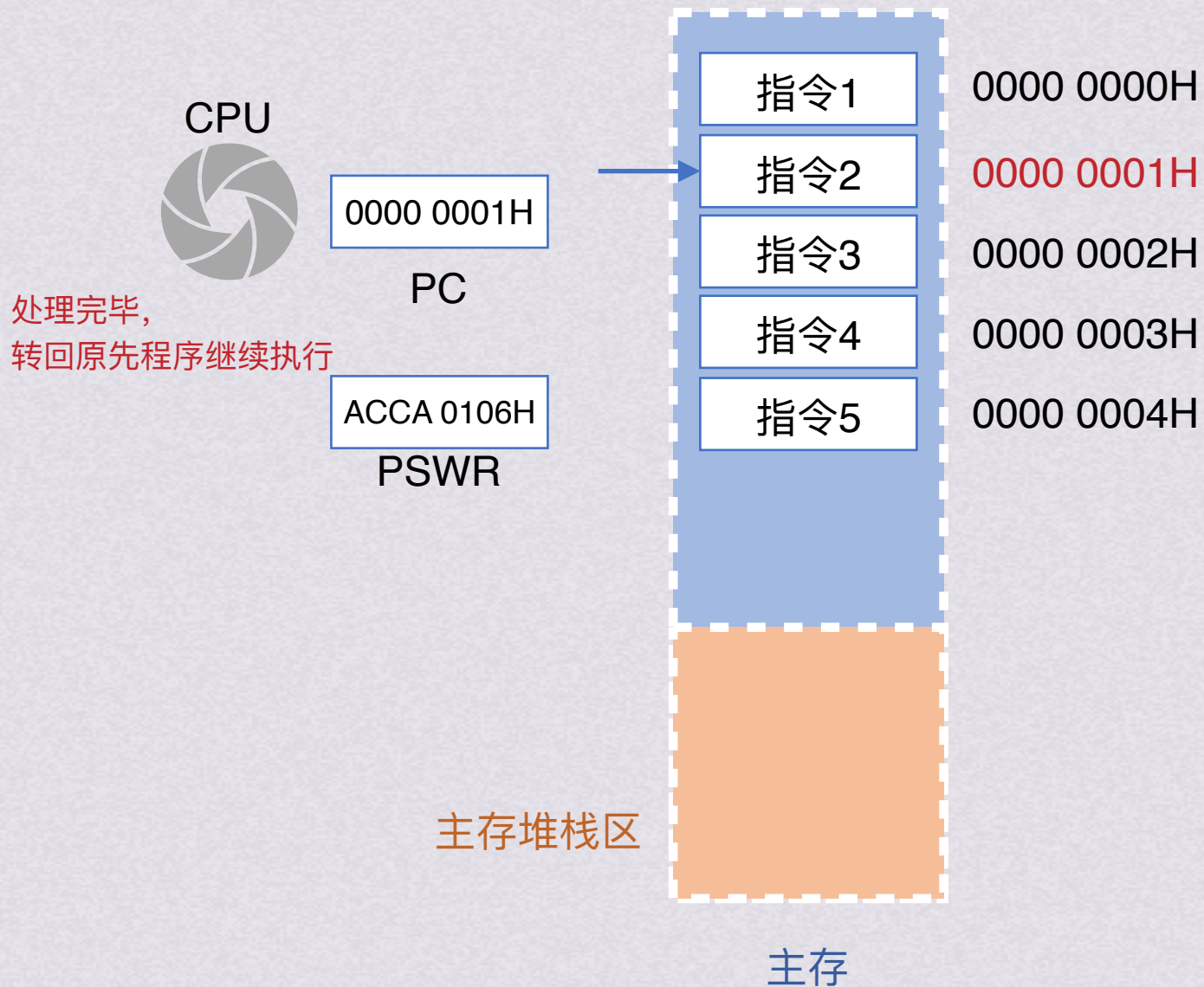
异常和中断处理

异常的处理过程

因为异常处理后可能还要回到原本被中断的程序继续执行，被中断时原程序的状态（如产生的各种标志信息、允许自陷标志等）都必须保存起来。

通常每个正在运行程序的状态信息被放在程序状态字寄存器（PSWR）中，存放在PSWR中的信息称为程序状态字PSW

与返回地址（断点）一样，PSW也要被保存到栈或特定寄存器中，在异常返回时，将保存的PSW恢复到PSWR中





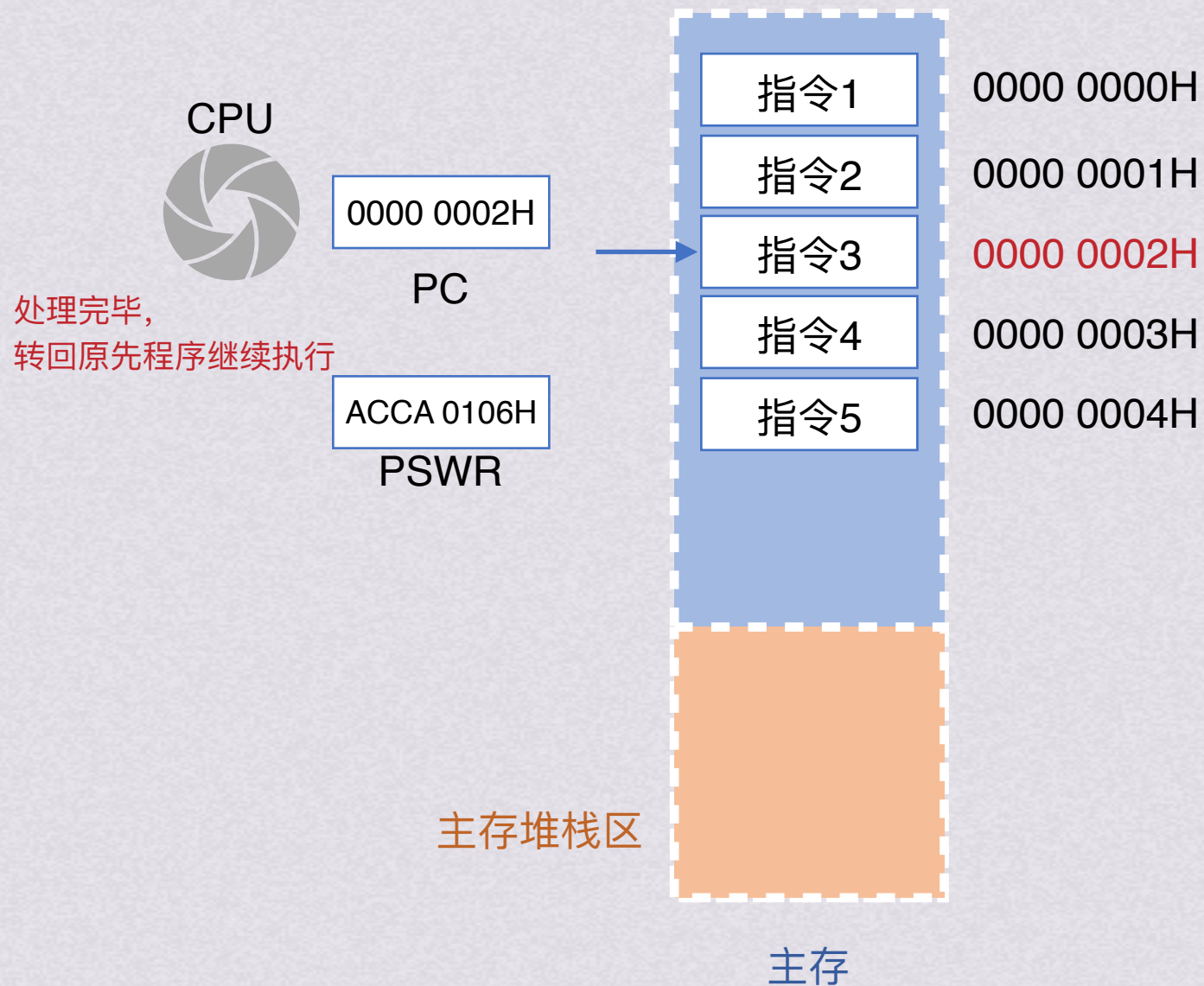
异常和中断处理

异常的处理过程

因为异常处理后可能还要回到原本被中断的程序继续执行，被中断时原程序的状态（如产生的各种标志信息、允许自陷标志等）都必须保存起来。

通常每个正在运行程序的状态信息被放在程序状态字寄存器（PSWR）中，存放在PSWR中的信息称为程序状态字PSW

与返回地址（断点）一样，PSW也要被保存到栈或特定寄存器中，在异常返回时，将保存的PSW恢复到PSWR中





异常和中断处理

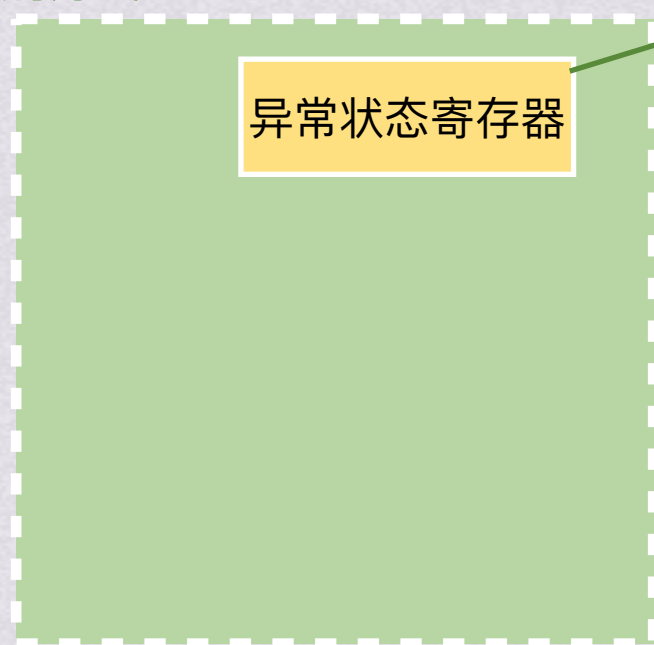
异常的处理过程

2.识别异常事件并转异常处理

必须知道发生了什么异常才能调出对应的异常处理程序，一般内部异常和外部中断源的识别方式不同，
大多处理器会将两者分开处理

内部异常事件的识别大多采用软件识别方式
外部中断源可以采用软件或者硬件识别方式

软件识别方式



用于记录异常原因

操作系统使用一个统一的异常查询程序，该程序按照一定优先级顺序查询异常状态寄存器，先查询到的异常先被处理

内部异常事件也可以不通过专门的查询程序来识别，而在发现异常时直接得到异常错误代码，根据不同的错误代码，转到相应的异常处理程序即可

CPU



异常和中断处理

异常的处理过程



如果在执行异常处理程序时又发生了新的可屏蔽中断怎么办？

为了处理新的可屏蔽中断，会把原来进程的现场、返回的断点和程序状态等破坏掉，
因此，应有一种机制禁止在响应并处理异常时再响应新的可屏蔽中断。

通常通过设置中断/异常状态位来实现

1则为开中断（允许中断）

0则为关中断

一般由操作系统通过管态指令来设置该位的状态



异常和中断处理

异常的处理过程



如果在执行异常处理程序时又发生了新的可屏蔽中断怎么办？

为了处理新的可屏蔽中断，会把原来进程的现场、返回的断点和程序状态等破坏掉，

因此，应有一种机制禁止在响应并处理异常时再响应新的可屏蔽中断。

通常通过设置中断/异常状态位来实现

1则为开中断（允许中断）

0则为关中断

一般由操作系统通过管态指令来设置该位的状态



CPU

允许中断开关: 0

PSWR/EFLAGS

控制是否允许中断



异常和中断处理

异常的处理过程



如果在执行异常处理程序时又发生了新的可屏蔽中断怎么办？

为了处理新的可屏蔽中断，会把原来进程的现场、返回的断点和程序状态等破坏掉，

因此，应有一种机制禁止在响应并处理异常时再响应新的可屏蔽中断。

通常通过设置中断/异常状态位来实现

1则为开中断（允许中断）

0则为关中断

一般由操作系统通过管态指令来设置该位的状态

取指周期

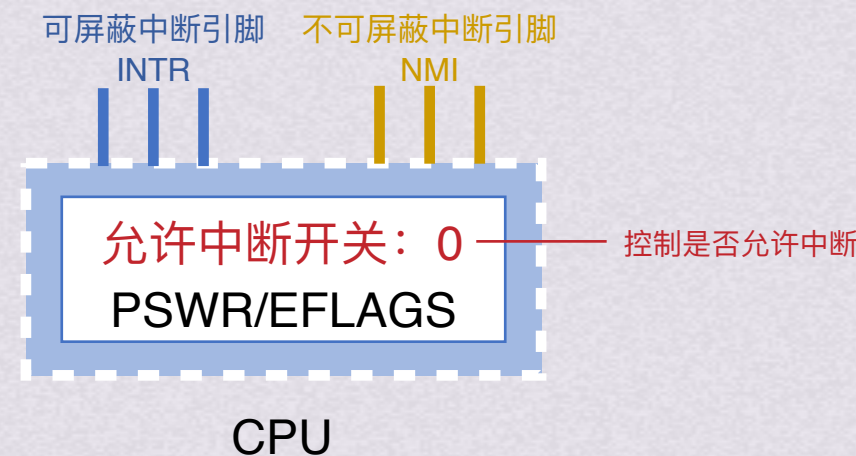
间址周期

执行周期

中断周期

指令周期

可屏蔽中断需要等到中断周期才能被检测到





小试牛刀

- 1.CPU只有在检测到中断请求信号后，才会进入中断响应周期
- 2.进入中断响应周期时，CPU一定处于中断允许（开中断）状态
- 3.CPU在执行一条指令时检测异常事件
- 4.CPU在执行完一条指令时检测中断请求信号
- 5.开中断时CPU检测到中断请求后就进行中断响应